MambaGuard: Certified Streaming Detection for Multi-Protocol LLM Agent Security

Anonymous Submission

Paper #XXXX, IEEE S&P 2027 Cycle 2

***Abstract*—Large language model (LLM) agents now communicate through four open protocols — the Model Context Protocol, the Agent-to-Agent Protocol, the Agent Communication Protocol, and the Agent Network Protocol — whose combined traffic forms a network-scale attack surface that prompt-boundary monitors and classical intrusion detection systems were not designed to observe. We present *MambaGuard*, a streaming detector that ingests a canonicalised protocol message graph, extracts per-agent sequential features with a selective state-space backbone, propagates relational context with a Lipschitz-normalised temporal graph attention layer, and selects defensive actions through a Hedge-based controller. The detector carries two complementary certificates. First, a randomised-smoothing certificate on the payload-embedding feature space yields a lower bound on the expected defender utility of the form $V^* - B\ \Pr[\neg\mathcal{C}_\varepsilon] - B\sqrt{\ln N/(2T)} - B\alpha_{MC}$, in which every term shares the utility unit so the composition type-checks. Second, a sparse-smoothing certificate on the protocol graph bounds the number of injected or removed edges an attacker can tolerate per window. Payload messages are encoded as *chunks* — one per tool description or short run of sentences — that the detector scores individually and pools per message (multiple-instance pooling). We evaluate on MCPTox, MCP-SafetyBench, AgentDojo, InjecAgent and a synthetic A2A sandbox with three seeds on grouped splits, against an episode-length control and a bag-of-words logistic-regression baseline, and add a *held-out-template* protocol in which the attack wordings or families of each test fold never appear in training. Chunk scoring is the single largest improvement we measured: against one embedding per message it raises macro-F1 from 0.50 to 0.70 on MCPTox, 0.50 to 0.88 on MCP-SafetyBench and 0.73 to 0.99 on AgentDojo. The detector separates the structural A2A attacks (macro-F1 0.952, AUROC 0.996) and exceeds the lexical baseline's AUROC on MCP-SafetyBench, but it does not beat the lexical baseline's macro-F1 on any benchmark, and under held-out templates both models degrade, the lexical baseline less so in AUROC. No architectural component is necessary on these benchmarks. We read this as a finding about the benchmarks as much as about detectors: their attacks are largely templated text, and an evaluation that holds out templates is needed before claiming generalisation. We also show that the block-product Lipschitz bound used in earlier detectors of this family is not a valid upper bound once residual paths are counted, and we replace it with a residual-aware bound that a gradient-ascent probe confirms. We release an anonymised reference implementation, the evaluation harness, and a reproducibility script that regenerates every table and figure from a single command.**

1. Introduction

Enterprise AI has moved from single-prompt interfaces to protocol-mediated agent meshes. A 2025 systematic review of the Model Context Protocol catalogued more than thirty attack techniques at the MCP layer alone [1], and the empirical MCPSecBench study reported attack success rates above eighty-five per cent on at least one major commercial deployment [2]. The MCP Inspector remote-code-execution flaw [3] and the A2ABreak catalogue of eleven protocol-level vulnerabilities [4] confirm that protocol exposure translates directly into system compromise. The defensive response so far has been fragmented along three separate axes.

First, *design-level* defences such as CaMeL [13] and MELON [14] prevent prompt-injection outcomes by sandboxing tool invocation or isolating untrusted content. These approaches are complementary to detection: they provide provable guarantees on specific agent decisions, not on the broader protocol traffic. Second, *tool-boundary monitors* like ClawGuard [9] enforce per-call deterministic policies, and program-dependency monitors like AgentArmor [8] reconstruct a per-trace dependency graph. Neither observes the multi-agent interaction graph, nor reports a quantitative robustness guarantee. Third, classical *network intrusion detection systems* have evolved through graph neural networks [28], transformer encoders [29], and selective state-space models [23] — but these operate on flat flow features and do not model MCP/A2A/ACP/ANP message semantics or certify their own output.

MambaGuard closes the gap between these three strands. It models the protocol stream as a time-stamped heterogeneous graph whose vertices are agents, tools, capability descriptors, and sessions, and whose edges carry canonicalised message tuples. A selective state-space backbone [19] processes each agent's message stream in linear time; a Lipschitz-normalised GATv2 layer [24], [26] propagates relational context; and a Hedge controller [37] selects defensive actions online with no-regret against the best fixed

commitment in hindsight. Two certificates accompany the detector: a randomised-smoothing certificate [30] on the payload-embedding feature space, and a sparse-smoothing certificate [34] on graph edges.

1.1. Contributions

- A **unified protocol-graph schema** for MCP, A2A, ACP and ANP (§4.1) with a five-class canonical message taxonomy.
- A **streaming composed detector** that couples a selective SSM with a Lipschitz-normalised temporal GATv2 layer over a per-window protocol graph (§4).
- A **dimensionally consistent two-certificate scheme** (§5): a Cohen-style smoothing certificate on feature-space perturbations with a composition theorem whose terms all carry the utility unit, plus a sparse-smoothing certificate on graph edits.
- **Chunk-level payload scoring** (§4.1): each message is split into tool- or sentence-level chunks that the detector scores and max/mean-pools, which keeps a single poisoned tool or injected sentence from being averaged away. It improves macro-F1 by 0.20–0.38 over one embedding per message on the three payload benchmarks (§7.2).
- A **measured study with controls** (§7): three seeds, bootstrap intervals, an episode-length control and a lexical baseline on every benchmark, plus component ablations. The detector matches the lexical baseline on structural A2A attacks and trails it in macro-F1 on payload-borne attacks; no architectural component is necessary on these benchmarks.
- A **held-out-template protocol and benchmark audit** (§7.3, §7.4): attack wordings and families are held out per fold. Top lexical features are the attack templates themselves; under held-out templates the lexical baseline's recall on MCPTox falls from 0.90 to 0.51 and the detector's macro-F1 from 0.73 to 0.67; leave-one-finding-out on the A2A sandbox is at chance for both models. Benign classes are small (3–24 test episodes on three benchmarks).
- An **anonymised artifact** that regenerates every table and figure with one command.

1.2. Scope and Non-Goals

The paper's guarantees are deliberately scoped. The feature-space certificate holds inside the payload-embedding feature space, not for arbitrary text edits; the graph certificate holds under a bounded number of edge additions or deletions, not under host compromise; and the online bound holds in the zero-sum restriction of the defender game. §5.6 and §9 state exactly what the certificates do and do not cover.

2. Background

Two technical ingredients underpin the paper. We state only what the body of the paper needs; extensions and standard proofs are deferred to Appendix A.

Selective state-space models.. A continuous state-space model maintains a hidden state $h(t) \in \mathbb{R}^N$ evolving as $h'(t) = Ah(t) + Bu(t)$, with output $y(t) = Ch(t)$. Zero-order-hold discretisation gives $h_t = \bar{A}h_{t-1} + \bar{B}u_t$, with $\bar{A} = \exp(\Delta A)$ and $\bar{B} = (\Delta A)^{-1}(\exp(\Delta A) - I)\Delta B$. The selective variant Mamba [19] makes Δ, B, C input-dependent and amortises inference in $\mathcal{O}(L)$ per sequence of length L via a hardware-aware parallel scan. We adopt the **S4D-Real** parameterisation [20], in which $A \in \mathbb{R}^N$ is a diagonal real matrix with strictly negative entries; this is the default Mamba initialisation and the choice reported to perform best on discrete modalities. For a real diagonal matrix the spectral radius equals the spectral (operator-2) norm, which is the property we exploit in the Lipschitz analysis of §5.2.

Randomised smoothing.. Given a base classifier f, the smoothed classifier $g(x) = \arg\max_c \Pr_{\delta\sim\mathcal{N}(0,\sigma^2 I)}[f(x+\delta) = c]$ is certifiably robust in ℓ_2 within a radius $r = \sigma\,\Phi^{-1}(p_A)$, where p_A is the probability of the top class under the noise distribution and Φ is the standard normal CDF [30]. In practice p_A is replaced by a Clopper-Pearson lower bound $\underline{p_A}$ estimated from n Monte-Carlo samples, with a failure probability α_{MC} absorbed into the certificate.

3. Threat Model and Security Goals

A robustness claim is only as meaningful as its threat model. We specify ours along four axes.

System model.. Several agents communicate through MCP, A2A, ACP and ANP. A trusted detector observes every protocol message, canonicalises it, and maintains a sliding-window protocol graph. A defender action is selected per round and applied to one message or session: deep packet inspection, blocking, session termination, agent isolation, or capability revocation.

Attacker knowledge.. We consider a *grey-box* attacker who knows the detector's architectural family (selective SSM plus temporal GATv2, protocol-graph representation, 34-class attack taxonomy), the published hyperparameters, and the payload encoder. The attacker does not have the deployed weights or the per-tenant Hedge random seed. For each experiment we also report a *white-box* setting in which the attacker additionally receives the trained weights and the embedding gradient; this is the standard worst case used by [39].

Attacker capabilities.. The attacker may (i) craft or rewrite protocol payloads inside a signed agent envelope; (ii) modify tool descriptions or Agent Cards; (iii) replay historical messages within a bounded time window; (iv) inject or delete graph edges up to a budget k per detection window through a compromised registered agent; (v) query the detector a bounded number of times to drive an attack search. The attacker cannot forge cryptographic signatures outside the compromised set, cannot compromise the host that runs the detector, and cannot modify the detector's

weights. Compromise of the model supply chain, memory, or host is explicitly out of scope.

Attacker action space $\mathcal{A}_A$.. Seven families from the agent-protocol attack literature [1], [2], [4]: tool poisoning, indirect prompt injection, rug-pull capability updates, capability shadowing, cross-server capability confusion, credential exfiltration, and sandbox bypass.

Perturbation budgets.. Two complementary budgets. $\varepsilon \geq 0$ bounds the ℓ_2 norm of perturbations at the frozen payload-embedding layer; this is where the smoothing certificate holds. $k \in \{0, 1, \ldots\}$ bounds the number of graph edges the attacker may add or delete inside one detection window; this is where the sparse-smoothing certificate holds. We do *not* claim that the ℓ_2 budget transports to a bound on raw text edits: the payload encoder has no meaningful global Lipschitz constant from token edits to embeddings, and a single token change can move an embedding arbitrarily far relative to ε. §5.6 returns to this restriction.

Security goals.. The defender seeks (G1) low per-message false-positive rate on benign traffic; (G2) high detection rate on malicious traffic; (G3) *certified* robustness within the stated perturbation budgets; (G4) sustained throughput at the message rate of the monitored deployment.

4. Protocol Graph and Detector

4.1. Unified Heterogeneous Protocol Graph

Let $\mathcal{G}(t) = (\mathcal{V}(t), \mathcal{E}(t), X(t))$ denote the protocol graph at wall-clock time t. The vertex set $\mathcal{V}(t) = \mathcal{V}_A \cup \mathcal{V}_T \cup \mathcal{V}_C \cup \mathcal{V}_S$ is partitioned into agents, tools or services, capability descriptors, and active sessions. Edges are time-stamped and typed; the five canonical types are agent-calls-tool, agent-messages-agent, agent-advertises-capability, tool-returns-data, and agent-delegates-task. Each protocol message is canonicalised into the tuple

$$m = (\tau, s, d, p, \mu, t_m),$$

where $\tau \in \{\mathcal{M}_{\text{tool}}, \mathcal{M}_{\text{comm}}, \mathcal{M}_{\text{cap}}, \mathcal{M}_{\text{data}}, \mathcal{M}_{\text{ctrl}}\}$, s, d are source/destination identifiers, $p \in \mathbb{R}^{d_p}$ is a payload embedding ($d_p = 1{,}280$: mean and maximum of frozen MiniLM chunk embeddings plus a hashed lexical vector, below), $\mu \in \mathbb{R}^{d_\mu}$ carries HTTP status, latency and auth metadata, and t_m is the message timestamp. A rule-based parser produces one tuple per raw protocol packet for each of MCP, A2A, ACP and ANP.

Chunk-level payload scoring.. A single embedding per message averages a poisoned tool description with every clean description in the same `tools/list` result, and an injected sentence with the rest of a long tool result. We therefore split each JSON payload into chunks $c_1, \ldots, c_K$: one per tool in a tool list (name, description and parameter descriptions), one per run of whole sentences of about 48 words in long text fields, and one structural header (method, keys, short values), with $K \leq 64$. Each chunk gets a frozen MiniLM embedding $e(c_k) \in \mathbb{R}^{384}$ and a hashed vector $\ell(c_k) \in \mathbb{R}^{512}$ of log word uni- and bigram counts, both L2-normalised. The message vector is $p = [\text{mean}_k\, e(c_k),\ \max_k e(c_k),\ \max_k \ell(c_k)]$, and the detector additionally scores every chunk with a two-layer network g and concatenates $[\max_k g(e(c_k), \ell(c_k)),\ \text{mean}_k\, g(\cdot)]$ to the input projection. Max pooling lets one chunk decide a coordinate (multiple-instance pooling); max and mean are both 1-Lipschitz, so the Lipschitz bound of §5.2 stays finite. The maximising chunk also localises a detection to a tool or sentence.

Worked example.. Consider a 2-second window in which Agent A issues an MCP `tools/list` to a database tool T_1 and receives a 400-byte JSON response; concurrently, Agent B sends an A2A `tasks/send` to Agent A delegating a user query, and a new capability descriptor C_3 is advertised through ANP by Agent C. The parser canonicalises six messages into three edges: $(A \to T_1, \mathcal{M}_{\text{tool}})$, $(T_1 \to A, \mathcal{M}_{\text{data}})$, $(B \to A, \mathcal{M}_{\text{comm}})$, and one capability-advertisement edge $(C \to C_3, \mathcal{M}_{\text{cap}})$. All four edges carry the same timestamp bucket $t_e \in [0, 2]$. The graph $\mathcal{G}(t)$ at the end of the window thus has six nodes (three agents, one tool, one capability, one session), four typed edges, and per-edge features comprising the 384-dim frozen embedding of the payload and a 32-dim metadata vector. This is the input the detector sees for one detection decision.

4.2. Selective SSM Backbone

For each agent $a \in \mathcal{V}_A$, the sequence of incident messages $x^a = (m_1^a, \ldots, m_{L_a}^a)$ is processed by a four-layer selective block (details in Appendix E):

$$h_t^a = \bar{A}_t\, h_{t-1}^a + \bar{B}_t\, m_t^a, \tag{1}$$

$$z_t^a = C_t\, h_t^a \odot \text{SiLU}\big(W_g\, m_t^a\big), \tag{2}$$

with the standard Mamba depthwise causal 1-D convolution applied to x^a before (1). The gated output $z_t^a \in \mathbb{R}^{d_z}$ is the per-agent sequential feature.

4.3. Lipschitz-Normalised Temporal GATv2

With $\{z^a\}_{a \in \mathcal{V}_A}$ available, a temporal GATv2 layer propagates relational information. For an ordered pair (a, a') joined by an edge with timestamp t_e, the attention logit is

$$e_{a,a'} = q^\top \text{LeakyReLU}\big(W_r\, [z^a \| z^{a'} \| e_{a,a'}^{\text{feat}} \| \Phi(t - t_e)]\big), \tag{3}$$

where $\Phi(\Delta t)$ is the Bochner-derived TGAT time encoding [25] with d_T learnable frequencies, $\|$ denotes concatenation, and q, W_r, W_v are learnable. We apply *LipschitzNorm* [26] to the attention weights; this is essential for the Lipschitz analysis below, because vanilla GATv2's dot-product attention is *not* Lipschitz for any ℓ_p norm on an unbounded domain (Kim *et al.* [27] prove this for self-attention and the argument transfers to GATv2). Multi-head attention with $H = 4$ heads is averaged at the final layer; the composed feature h^a feeds a softmax head over the 34-class attack taxonomy.

Algorithm 1 MambaGuard online defence loop

Require: Horizon T, utility range B, number of defender actions $N = |\mathcal{A}_D|$

$\eta \leftarrow \sqrt{8 \ln N / T} \,/\, B$ ▷ Corrected tuning (§5.4)
$w_1(s) \leftarrow 1$ for all $s \in \mathcal{A}_D$
for $t = 1, \ldots, T$ **do**
  Observe $\mathcal{G}(t)$ and per-agent streams
  Compute z_t^a via (1)–(2)
  Compute h_t^a via (3)
  Score classes $\hat{y}_t \leftarrow \mathrm{softmax}(W_h h_t)$
  Sample action $s_t \sim p_t(s) = w_t(s) / \sum_{s'} w_t(s')$
  Observe loss $\ell_t(s) \in [0, B]$
  $w_{t+1}(s) \leftarrow w_t(s) \exp(-\eta\, \ell_t(s))$
end for

4.4. Hedge Controller

The detector logits parameterise a Hedge [37] selector over the five defensive actions $\mathcal{A}_D$. Algorithm 1 states the complete loop.

5. Certified Guarantees

The detector carries two certificates — feature-space (§5.1) and graph-edit (§5.3) — plus a no-regret guarantee on the Hedge controller (§5.4). §5.5 combines them in a dimensionally consistent composition theorem, and §5.6 states what the certificates do and do not operationally guarantee.

5.1. Feature-Space Smoothing Certificate

We define the smoothed detector $g(x) = \arg\max_c \mathbb{P}_{\delta \sim \mathcal{N}(0, \sigma^2 I)}[f(x + \delta) = c]$ over the payload-embedding input x. Following [30], for a given x let $\underline{p_A}$ be the Clopper-Pearson lower bound on $\mathbb{P}[f(x + \delta) = \overline{g(x)}]$ from n Monte-Carlo samples at confidence $1 - \alpha_{MC}$. Then with probability at least $1 - \alpha_{MC}$ the certified radius is

$$R(x) = \sigma\, \Phi^{-1}\big(\underline{p_A}\big), \tag{4}$$

and $g(x + \delta') = g(x)$ for all $\|\delta'\|_2 \le R(x)$. We report certified-accuracy-vs-radius curves rather than a single number, because the radius varies per input (§7.5).

5.2. Local Lipschitz Bound on the Composed Detector

The smoothed detector's accuracy is tightened when the base detector is Lipschitz on the perturbed domain. For MambaGuard the base map is $f = h \circ g_{\text{gat}} \circ g_{\text{ssm}}$. We state the bound under explicit bounded-input assumptions, because the selective SSM and the gated output are both products of input-dependent factors and therefore *not* globally Lipschitz on $\mathbb{R}^d$.

Assumption 1 (Bounded operating regime)**.** *(i) Inputs are projected to a Euclidean ball $\|x_t\|_2 \le X_{\max}$; (ii) discretisation steps are clamped to $\Delta_t \in [\Delta_{\min}, \Delta_{\max}]$; (iii) spectral norms of the projections producing B_t, C_t and Δ_t (including their Jacobians) are bounded by M_B, M_C, M_Δ respectively; (iv) the state matrix $A \in \mathbb{R}^N$ is real diagonal with strictly negative entries (S4D-Real) and $\Delta_{\max} \max_i A_i < 0$, so $\rho(\bar{A}_t) = \|\bar{A}_t\|_2 \le \rho < 1$.*

Lemma 1 (Local Lipschitz of the selective block)**.** *Under Assumption 1, with $\beta := \sup_t \|\bar{B}_t\|_2$, $\gamma := \sup_t \|C_t\|_2$, $\kappa_g := \|W_g\|_2$, $L_\sigma \approx 1.0998$ the exact SiLU Lipschitz constant, and $X_{\max}$ the input-norm cap, the selective block g_{ssm} is Lipschitz on $\{x : \|x_t\| \le X_{\max}\}$ with*

$$L_{ssm} \le \frac{\gamma\beta(1 + L_\sigma \kappa_g)}{1 - \rho} + \frac{\gamma\beta X_{\max}(M_B + M_C + M_\Delta)}{(1 - \rho)^2}. \tag{5}$$

The second term accounts for the input-dependent factors in $\bar{B}_t, C_t, \Delta_t$; it vanishes only when those parameters are input-independent (the S4 limit). Proof in Appendix B.

Lemma 2 (LipschitzNormed GATv2)**.** *With LipschitzNorm scaling constant η and Bochner frequencies $\{\omega_k\}$, the temporal GATv2 layer (3) satisfies*

$$L_{gat} \le \|W_v\|_2 \big(1 + \eta\, \|q\|_2\, \|W_r\|_2\big), \tag{6}$$

$$L_\Phi \le \|W_v\|_2\, \eta\, \|q\|_2\, \|W_r\|_2 \sqrt{\tfrac{2}{d_T} \textstyle\sum_k \omega_k^2}. \tag{7}$$

Residual paths matter.. The detector stacks S residual SSM blocks after an input projection W_{in}, so the sound composition is

$$L_f \;\le\; M_h\, L_{\text{gat}}\, \|W_{\text{in}}\|_2 \prod_{\ell=1}^{S} \big(1 + L_{\text{ssm}}^{(\ell)}\big). \tag{8}$$

The plain product $M_h L_{\text{gat}} \prod_\ell L_{\text{ssm}}^{(\ell)}$, which drops the identity branch of every residual block and the input projection, is *not* an upper bound: in §7.5 a gradient-ascent probe exceeds it by a factor of 3.6. The smoothing certificates we report (§7.5) do not depend on (8); the bound is a diagnostic of the architecture, and **we do not claim it transfers to raw text perturbations**.

5.3. Sparse-Smoothing Graph-Edit Certificate

A continuous certificate does not express structural attacks such as Agent Card impersonation, which add or remove edges in $\mathcal{G}(t)$. We apply the sparse-smoothing framework of Bojchevski *et al.* [34]. For edge-flip probabilities p_+ (adding an absent edge) and p_- (deleting a present edge), the smoothed classifier on edge-perturbed graphs is certified robust to budgets (k_a, k_d) whenever

$$\sum_{\mathbf{Q} \in \mathcal{H}} \min\{\mathbf{p}_{\mathbf{Q}}^{+}, \mathbf{p}_{\mathbf{Q}}^{-}\} \;\ge\; p_A - 0.5, \tag{9}$$

where the pointwise bound is taken over the Neyman-Pearson regions $\mathcal{H}$ induced by budgets k_a edge additions and k_d edge deletions, and $\mathbf{p}^{\pm}$ are the per-region likelihoods under the clean and perturbed noise distributions (full statement in [34], Theorem 1). In practice we evaluate the bound over the attacker's candidate edge set, not over all node pairs, which both reduces variance and matches the registered-agent restriction in the threat model.

5.4. Online No-Regret Guarantee

For losses in $[0, B]$ the Hedge regret bound $R_T \leq (\ln N)/\eta + \eta T B^2/8$ is minimised at

$$\eta^* \;=\; \sqrt{8 \ln N / T} \;/\; B, \tag{10}$$

giving

$$R_T \;\leq\; B\sqrt{(T/2)\ln N}. \tag{11}$$

(This is the classical Cesa-Bianchi–Lugosi tuning [37]; the frequently quoted $\eta = \sqrt{2\ln N/T}$ minimises the cruder bound $(\ln N)/\eta + \eta T/2$ and gives $\sqrt{2T\ln N}$, twice the constant.) For an anytime variant with $\eta_t = \sqrt{8\ln N/t}/B$, the regret is at most $B(\sqrt{2T\ln N} + \ln N/8)$.

Theorem 1 (No-regret)**.** *Algorithm 1 with η as in (10) satisfies (11).*

We do *not* claim that no-regret dynamics converge to the Stackelberg leader value in general. That claim is only true in two restricted settings: (a) the defender-attacker game is *zero-sum*, in which case the Stackelberg value equals the minimax value and Hedge's time-average payoff is within R_T/T of it; or (b) the guarantee is phrased as regret against the *best fixed commitment in hindsight* over a finite set of attacker types [38], with the action set restricted to the extreme points of the attacker best-response polytopes. Our experiments report both views.

5.5. Composed Certificate (Type-Checked)

Let $\mathcal{C}_\varepsilon$ be the event that the smoothed detector's prediction is certified constant at radius ε, as in (4). On $\mathcal{C}_\varepsilon$ the defender's action and utility are unchanged under attack; off $\mathcal{C}_\varepsilon$ the per-round loss is at most B. Combining with Theorem 1 gives the composition.

Theorem 2 (Composed certificate)**.** *Let V^* be the Stackelberg value under the restriction of §5.4(a) or (b), $\hat{\pi}_T$ the Hedge policy after T rounds, $\varepsilon \geq 0$ the feature-space ℓ_2 budget, and α_{MC} the Monte-Carlo failure probability in (4). Then*

$$\mathbb{E}\big[V_{ach}(\hat{\pi}_T, \varepsilon)\big] \;\geq\; V^* - B\,\mathbb{P}\big[\neg\mathcal{C}_\varepsilon\big] - B\sqrt{\tfrac{\ln N}{2T}} - B\,\alpha_{MC}. \tag{12}$$

All four terms share the utility unit; $\mathbb{P}[\neg\mathcal{C}_\varepsilon] = 1 - \mathrm{CertAcc}(\varepsilon)$ is read off the certified-accuracy curve. Proof in Appendix C.

5.6. Operational Meaning and Non-Guarantees

Theorem 2 guarantees a lower bound on long-run expected defender utility against any attacker whose per-round perturbation stays within the stated budgets. It does *not* guarantee: (i) that an arbitrary text edit of a payload lies within ε in embedding space (the encoder has no global token-to-embedding Lipschitz constant); (ii) robustness to graph edits beyond $k_a + k_d$ within one window; (iii) robustness to host compromise, cryptographic-signature forgery, or zero-day protocol-implementation flaws. Applications that require payload-level provable robustness should additionally use the erase-and-check certificate of Kumar *et al.* [33] against suffix/insertion/infusion up to d tokens; our evaluation reports this composition in §7.5.

6. Experimental Setup

6.1. Benchmarks

We evaluate on six benchmarks that span the agent-protocol attack surface. None of them records protocol traffic with timestamps, so each case is rendered as an *episode*: the JSON-RPC messages an observer between host and servers would see around one user task, with a per-message label marking attacker-controlled content (construction, pinned source commits and counts in Appendix F). Splits are grouped and stratified so that no server, case or attack/benign pair appears in two splits.

MCPTox [6]: 1,312 tool-poisoning instances over 10 risk categories on 45 real MCP servers (353 tools). Each attack episode lists the server's clean tools plus one poisoned tool; 533 benign episodes replay the same user queries with the clean tool list. Split by server (train/val/test 1,263/283/299 episodes), so test servers are unseen.

MCP-SafetyBench [5]: 245 attack cases over 5 domains and 20 attack types. Clean tool lists are taken from the benchmark's own server sources; each attack is placed on the message that carries it (poisoned description, added tool, injected or forged tool result, replayed or spoofed call, or the user request). We use the 238 cases whose attack is visible on the wire (the 7 rug-pull cases change tool code but no message) plus 162 exact benign twins for the description-borne attacks.

MCPSecBench [2]: the 11 attacks for which the benchmark provides prompts, rendered against its benign and malicious playground servers, with 7 benign twins. Because the twins differ in the number of servers, episode length alone separates the classes, so we report only message-level detection on this set.

A2A-sandbox: synthetic A2A v1.0 traces we generate for the 11 vulnerability classes of A2ABreak [4], whose artifact analyses the specification and contains no traffic. Each of 2,200 attack episodes has a benign twin with the same participants and task that differs only in the decisive field (requested scope, reused `contextId`, webhook host, chunk index, ...); both receive identical random background exchanges. We report results on the default pair-grouped split and leave-one-finding-out.

AgentDojo [11]: the recorded agent runs shipped with the benchmark (no agent is re-run) for one model (`claude-3-5-sonnet`); a tool result is labelled as attacker-controlled when the injected string surfaces in it. 550 attack and 97 benign episodes. **InjecAgent** [12]: 1,054 templated cases, each a tool result whose single injection slot is filled with the attacker instruction; benign twins fill the slot with neutral text.

Shortcut control.. For every benchmark we also report a one-threshold classifier on episode length. It reaches balanced test accuracy 0.50 on MCPTox, MCP-SafetyBench, AgentDojo and InjecAgent, and 0.53 on the A2A-sandbox, so a detector must beat these numbers to be doing anything beyond counting messages. **Lexical baseline.** We also train a TF–IDF logistic regression on the concatenated JSON text of each episode (word uni- and bigrams, minimum document frequency 2, at most 5×10^4 features, balanced class weights, $C = 1$) on the same splits, with the same validation-tuned threshold. It has no sequence model, no graph and no protocol knowledge, so it measures how much of a benchmark words alone solve.

Small benign classes. The test splits contain 330 (A2A), 70 (MCPTox), 24 (MCP-SafetyBench), 14 (AgentDojo) and 3 (InjecAgent) benign episodes; false-positive rates on the last three rest on very few windows.

6.2. Payload Encoder, Metrics and Protocol

Payloads are encoded with the chunk scheme of §4.1 using a frozen `all-MiniLM-L6-v2` encoder. The scheme was chosen among four candidates (one embedding per message; chunk mean+max; chunk mean+max with the lexical vector; chunk scoring) on MCPTox *validation* macro-F1 in a CPU pilot, before any test result of the GPU study existed; the encoder ablation (§7.2) reports all four. All detector runs use the per-window protocol-graph read-out, four selective-SSM blocks, two GATv2 layers and the chunk scorer (4.10 M trainable parameters), AdamW (learning rate 3×10^{-4}, 200 warm-up steps), focal loss, the Lipschitz regulariser $\lambda_L = 10^{-3}$, batch 32, class-balanced sampling of training windows, and early stopping on validation macro-F1 with patience 5 (at most 15, 20, 30, 30 and 20 epochs for A2A, MCPTox, MCP-SafetyBench, AgentDojo and InjecAgent). Each configuration is trained with seeds 0, 1 and 2 (ablations and held-out folds: seeds 0 and 1; A2A held-out folds: seed 0) in fp32 on one NVIDIA T4; the whole study, including ablations, held-out folds, certificates and latency, took 7.1 GPU-hours (Appendix E).

Held-out templates. To test whether a model recognises reworded attacks rather than memorised text, each fold removes one attack wording or family from training and validation and keeps only those attacks, plus the benign episodes of the test groups, in the test split. Folds: MCPTox by wrapper style (`important_tag`, `plain`, `ignore_previous`) and, separately, by attack paradigm (Template-1/2/3); AgentDojo by attack family (`important_instructions` and its five variants, `tool_knowledge`, `ignore_previous`, `injecagent`, `direct`, and the five denial-of-service attacks), using the recorded GPT-4o runs, the only ones attacked with many templates, capped at 100/25/40 attack episodes per family for train/val/test; InjecAgent by attacker case (4 hashed groups); MCP-SafetyBench by attack family (tool poisoning, name competition, result injection, host/user); and the A2A sandbox by vulnerability class (leave-one-finding-out, 100 pairs per finding). We verified for every fold that no held-out attack appears in training or validation and that no episode or group crosses splits. The lexical baseline is trained on exactly the same folds.

The decision threshold is chosen on the validation split to maximise macro-F1 and then applied to the test split. We report test macro-F1 at that threshold (primary), macro-F1 at 0.5, AUROC, attack recall and false-positive rate, as mean $\pm$ standard deviation over seeds, and a 1,000-sample bootstrap 95% interval for seed 0. A head-to-head comparison with the published agent defences of §10 needs each one retrained on the same splits and is not part of this study.

TABLE 1. TEST DETECTION, MEAN $\pm$ S.D. OVER THREE SEEDS (CHUNK-SCORING ENCODER, ONE T4). MACRO-F1 USES THE THRESHOLD TUNED ON VALIDATION. "LEXICAL" IS THE TF–IDF LOGISTIC-REGRESSION BASELINE ON THE SAME SPLITS AND THRESHOLD RULE; "LEN." IS THE BALANCED ACCURACY OF THE EPISODE-LENGTH CONTROL. n: TEST EPISODES (BENIGN IN BRACKETS). MCPSECBENCH (18 EPISODES WHOSE CLASSES DIFFER IN LENGTH) SUPPORTS ONLY MESSAGE-LEVEL EVALUATION AND IS NOT INCLUDED.

		MambaGuard		Lexical		
Benchmark	n **(ben.)**	**Macro-F1**	**AUROC**	**F1**	**AUROC**	**Len.**
A2A sandbox	660 (330)	$0.952_{\pm.002}$	0.996	0.955	0.995	0.53
MCPTox	265 (70)	$0.731_{\pm.095}$	0.850	**0.849**	0.917	0.50
MCP-SafetyBench	59 (24)	$0.844_{\pm.068}$	**0.954**	0.880	0.935	0.50
AgentDojo	104 (14)	$0.953_{\pm.064}$	0.990	1.000	1.000	0.50
InjecAgent	189 (3)	$0.966_{\pm.059}$	1.000	1.000	1.000	0.50

7. Results

7.1. Per-Benchmark Detection

Table 1 reports test detection for the detector and both controls. On the A2A sandbox, where the attack is *structural* — a reused context identifier, an unverified webhook host, an over-claimed Agent Card, an out-of-order task state — MambaGuard reaches macro-F1 0.952 ± 0.002 and AUROC 0.996, far above the 0.53 length control and level with the lexical baseline (0.955). Where the attack is a text payload, chunk scoring gives macro-F1 0.73 on MCPTox (AUROC 0.85), 0.84 on MCP-SafetyBench (AUROC 0.95, above the lexical baseline's 0.935), 0.95 on AgentDojo (AUROC 0.99) and 0.97 on InjecAgent (AUROC 1.00). The lexical baseline is still higher in macro-F1 everywhere (0.85, 0.88, 1.00, 1.00), and the detector's seed variance is large on MCPTox (± 0.095, driven by seed 1, whose validation-tuned threshold of 0.21 gave a false-positive rate of 0.74; seeds 0 and 2 reach 0.78 and 0.79). The seed-0 bootstrap intervals are $[0.72, 0.83]$ on MCPTox and $[0.78, 0.96]$ on MCP-SafetyBench. InjecAgent's three benign test windows make its false-positive rate uninformative.

7.2. Encoding Payloads: Chunk Scoring

Table 2 isolates the payload encoder on the three payload benchmarks (seeds 0 and 1, everything else fixed).

TABLE 2. PAYLOAD-ENCODER ABLATION: TEST MACRO-F1 / AUROC, MEAN OVER SEEDS 0 AND 1. ROWS ADD, IN ORDER, CHUNKING, THE PER-CHUNK LEXICAL VECTOR AND LEARNED CHUNK SCORING.

Encoder	MCPTox	MCP-SafetyBench	AgentDojo
One MiniLM vector per message	0.502 / 0.574	0.499 / 0.499	0.732 / 0.916
Chunks, mean+max	0.587 / 0.618	0.588 / 0.630	0.910 / 0.998
+ lexical vector	0.603 / 0.646	0.689 / 0.741	0.854 / 0.994
+ chunk scoring (ours)	**0.700 / 0.854**	**0.883 / 0.973**	**0.989 / 1.000**

One MiniLM embedding per message is at chance on MCPTox and MCP-SafetyBench (macro-F1 0.50, AUROC 0.50–0.57). Pooling chunk embeddings helps a little (0.59), adding the lexical vector a little more (0.60–0.69), but the large step comes from letting the detector score each chunk before pooling: macro-F1 0.70 on MCPTox, 0.88 on MCP-SafetyBench and 0.99 on AgentDojo, with AUROC rising to 0.85, 0.97 and 1.00. The attack occupies one chunk among several (5.5 chunks per message on average in MCPTox, 6.0 in MCP-SafetyBench), and only a pooling that can be decided by one chunk recovers it.

7.3. What a Lexical Baseline Reveals

The bag-of-words classifier is as good as or better than every detector variant in macro-F1 (Table 1). We checked that this is not leakage through identifiers: its heaviest attack features are the attack text itself — "important", "you must", "ignore the previous", "when using" on MCPTox; the fixed AgentDojo injection template ("before you can solve the task that I gave you"); the attacker's request on InjecAgent. Benign classes are small and narrow: InjecAgent's 17 benign episodes all carry one filler sentence, MCP-SafetyBench's benign twins exist only for description-borne attacks and come mostly from the web-search servers ("serper" and "serpapi" are among the strongest benign features), and the A2A sandbox's attack templates have their own vocabulary ("jwks", "admin", "hr records"), which likely explains why the lexical baseline also reaches 0.95 there. Episode-level scores on these corpora therefore measure, to a large extent, recognition of a small set of attack templates. The held-out protocol below tests what remains when the template is new.

7.4. Held-Out Templates

Table 3 reports the mean over held-out folds. Every model loses accuracy when the attack wording is new, but by different amounts and in different ways.

MCPTox. The detector's macro-F1 falls from 0.73 to 0.67 and the lexical baseline's from 0.85 to 0.70; the baseline keeps the higher AUROC (0.87 vs 0.74). At the validation-tuned threshold the two behave differently: the lexical baseline becomes conservative (attack recall 0.51, false-positive rate 0.07; recall 0.29 and 0.32 on the unseen `important_tag` and `plain` wrappers), while the detector keeps recall at 0.74 at a false-positive rate of 0.37. Per fold, the detector wins macro-F1 on Template-3 (0.76 vs 0.57) and `plain` (0.65 vs 0.62), ties on `important_tag` (0.57), and loses on the other three.

TABLE 3. HELD-OUT TEMPLATES: TEST MACRO-F1 / AUROC, MEAN OVER FOLDS (FOLDS IN §6.2; SEEDS 0–1, A2A SEED 0), WITH THE IN-DISTRIBUTION MACRO-F1 FROM TABLE 1 FOR REFERENCE. RECALL AND FPR AT THE VALIDATION-TUNED THRESHOLD.

	MambaGuard			Lexical		
Benchmark (folds)	F1 (in-dist.)	AUROC	Rec./FPR	F1 (in-dist.)	AUROC	Rec./FPR
MCPTox (6)	0.670 (0.731)	0.743	0.74/0.37	0.698 (0.849)	0.870	0.51/0.07
AgentDojo (6)	0.770 (0.953)	0.895	0.82/0.23	0.902 (1.000)	0.981	0.96/0.18
MCP-SafetyBench (4)	0.480 (0.844)	0.563	0.52/0.43	0.560 (0.880)	0.670	0.19/0.05
A2A sandbox (11)	0.387 (0.952)	0.527	0.28/0.30	0.447 (0.955)	0.606	0.31/0.20
InjecAgent (4)	0.928 (0.966)	1.000	0.99/0.08	0.955 (1.000)	1.000	0.99/0.00

AgentDojo. Holding out an attack family costs the detector 0.18 macro-F1 (0.95 → 0.77, AUROC 0.90) and the lexical baseline 0.10 (1.00 → 0.90, AUROC 0.98). The hardest family for both is `direct`, which states the attacker's goal without any of the wrapper text the other families share.

MCP-SafetyBench and the A2A sandbox. With an attack family or vulnerability class held out, both models are near chance: MCP-SafetyBench macro-F1 0.48 (detector) and 0.56 (lexical), A2A leave-one-finding-out 0.39 and 0.45, with AUROC 0.53 and 0.61. On A2A this is expected: each finding is a different protocol violation, and nothing in the other ten tells a model what the eleventh looks like. *InjecAgent* stays saturated for both (AUROC 1.00) because unseen attacker cases still share the injection slot and the single benign filler.

We draw two conclusions. First, neither model generalises across attack *families*; claims of generalisation on these benchmarks need family-level held-out evaluation. Second, at a fixed, validation-chosen threshold, the detector trades precision for recall on reworded attacks, while the lexical baseline keeps precision but misses about half of them; threshold-free (AUROC), the lexical baseline remains ahead.

7.5. Certification: Actual Lipschitz Constant, Smoothing and Graph-Edit

Table 4 reports the per-component Lipschitz factors as a function of the effective spectral cap $c = \sup_\ell \|W^{(\ell)}\|_2$ enforced by the regulariser λ_L. The key finding is at $c = 0.5$: the block-product quantity is 0.023, but projected gradient ascent on δ finds a ratio $\|f(x+\delta) - f(x)\|/\|\delta\|$ of 0.083, i.e. 3.6× larger. The block product is therefore *not* an upper bound; it omits the identity branch of each residual block and the input projection. The residual-aware bound (8) is 3.09, which the probe respects, and which is 37× looser than the probe. Tighter LipSDP-style estimates [32] are future work. On the *trained* benchmark models the residual-aware bound is vacuous: 7×10^4–5×10^6 for the full model with the regulariser ($\lambda_L = 10^{-3}$) and 3.6×10^8 without it on A2A (Table 6). The regulariser lowers the bound by almost two orders of magnitude, but neither value certifies anything;

TABLE 4. LIPSCHITZ BOUNDS VERSUS THE EFFECTIVE SPECTRAL CAP c IMPOSED ON EVERY LINEAR LAYER (DEFAULT 3.68 M-PARAMETER CONFIGURATION). "BLOCK PRODUCT" IS $M_h L_{\text{GAT}} \prod_\ell L_{\text{SSM}}^{(\ell)}$; "RESIDUAL" IS THE SOUND BOUND (8); $\hat{L}_f$ IS THE EMPIRICAL LOWER BOUND FROM 200-STEP PROJECTED GRADIENT ASCENT ON δ ("—" WHERE THE RESIDUAL BOUND AND THE PROBE WERE NOT EVALUATED; THEY WERE RUN AT $c = 0.5$).

c	L_{ssm}	L_{gat}	Block product	Residual	$\hat{L}_f$
0.50	0.06	1.46	0.023	3.09	0.083
0.75	12.88	5.50	39.87	—	—
1.00	58.84	14.73	651.80	—	—
1.25	89.74	32.30	2482.52	—	—
2.00	177.96	67.99	10364.17	—	—

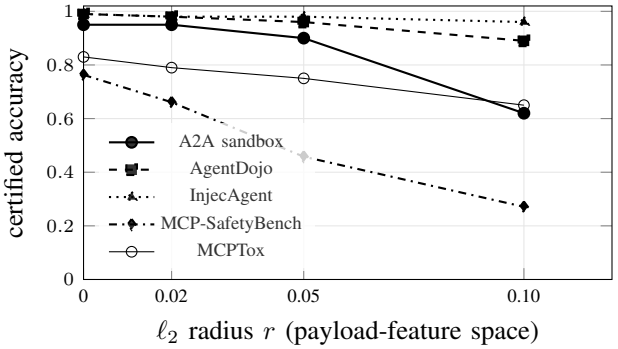


Figure 1. Measured certified accuracy versus ℓ_2 radius on the benchmark test sets (seed-0 models, $\sigma = 0.05$, $n = 500$, $\alpha_{MC} = 10^{-3}$). The maximum certifiable radius at this n and σ is ≈ 0.11.

the certificates below therefore use randomised smoothing alone.

Figure 1 plots certified accuracy against radius for the seed-0 model of each benchmark: the fraction of test windows that the smoothed detector classifies correctly *and* certifies at radius r. Gaussian noise is added to the detector's whole payload input, the pooled message vector and every chunk feature (100 test windows, 59 for MCP-SafetyBench; $n_0 = 32$ selection and $n = 500$ estimation samples; $\alpha_{MC} = 10^{-3}$). With $n = 500$ the largest certifiable radius is $\sigma\Phi^{-1}(\alpha_{MC}^{1/n}) \approx 2.2\sigma$. At $\sigma = 0.05$, certified accuracy at $r = 0.05$ is 0.90 on A2A, 0.75 on MCPTox, 0.96 on AgentDojo, 0.98 on InjecAgent and 0.46 on MCP-SafetyBench; feeding the curves into Theorem 2 ($V^* = B = 1$, $T = 10^4$, $N = 5$) gives defender-utility lower bounds of 0.89, 0.74, 0.95, 0.97 and 0.45. A larger σ buys radius on the payload benchmarks: at $\sigma = 0.25$, certified accuracy at $r = 0.25$ is 0.48 on MCPTox, 0.78 on InjecAgent and 0.21 on AgentDojo, while on A2A it costs accuracy (abstention 0.32). The InjecAgent curve is high partly because 98% of its windows are attacks. The radii are measured on L2-normalised embeddings; we have not measured how far a text edit moves an embedding, so none of them is a text-level guarantee (§5.6).

For the *graph-edit* certificate we use sparse smoothing with $p_+ = p_- = 0.05$, restricted to the attacker's candidate edge set of registered agents. As a correctness check of the implementation, on a 10-node test graph with $n = 1000$ smoothing samples and $\alpha = 10^{-3}$ the certificate returns $\underline{p_A} = 0.993$ and a budget of $(k_a, k_d) = (2, 2)$. A benchmark-scale evaluation of certified edge budgets is not part of this study.

TABLE 5. LATENCY ON ONE T4. DETECTOR: MEDIAN OF 100 FORWARD PASSES AFTER 10 WARM-UPS ON PRE-COMPUTED PAYLOAD FEATURES; "BATCH 1" IS ONE DETECTION WINDOW (3–8 MESSAGES); "BEST BATCH" IS THE NUMBER OF WINDOWS (1, 8, 32 OR 128) THAT MAXIMISES MESSAGES PER SECOND. ENCODER: CHUNKING, MINILM AND LEXICAL HASHING PER MESSAGE, ONE MESSAGE AT A TIME AND IN BATCHES OF 200.

	Detector, batch 1		**Detector, best batch**		**Encoder (ms/msg)**	
Benchmark	**median**	**p99**	**per msg**	**msg/s**	**one**	**batched**
A2A sandbox	9.8 ms	11.5 ms	0.12 ms	8,644	8.6	1.59
MCPTox	9.4 ms	11.4 ms	0.12 ms	8,557	19.4	1.73
MCP-SafetyBench	9.5 ms	10.8 ms	0.18 ms	5,644	9.5	3.08
AgentDojo	8.4 ms	9.8 ms	0.20 ms	5,127	8.7	1.87
InjecAgent	9.4 ms	10.9 ms	0.10 ms	10,121	8.8	1.13

7.6. Latency and Throughput, Harmonised

Per-message latency and aggregate throughput are different quantities, and a single number cannot stand for both. Table 5 therefore reports (i) the streaming batch-1 latency of one detection window, (ii) the amortised per-message cost of the detector at the batch size that maximises throughput, and (iii) the cost of the chunked payload encoder, measured separately. All timings are on one T4 with our reference (non-fused) selective scan.

A detection window costs about 9–10 ms one at a time, dominated by fixed per-call overhead; batching windows brings the detector to 0.10–0.20 ms per message (5,000–10,000 messages per second). The payload encoder is now the bottleneck: 1.1–3.1 ms per message in batches, i.e. 325–887 messages per second end to end on one T4, and 9–19 ms for a single message (MCPTox tool lists have the most chunks). We make no claim for a fused-kernel or optimised-encoder deployment, which we did not measure.

7.7. Component Ablation

Table 6 switches off one component at a time on the A2A sandbox (structural attacks) and on MCPTox (payload attacks), with the chunk-scoring encoder throughout (two seeds per variant, three for the full model). On the A2A sandbox every variant reaches macro-F1 0.952–0.961 and AUROC 0.996–0.997; the differences are within the seed spread. On MCPTox the full model is the *weakest* variant: the flat read-out, which drops the protocol graph and treats the window as one sequence, reaches 0.841 macro-F1 and 0.945 AUROC — level with the lexical baseline's macro-F1 (0.849) and above its AUROC (0.917) — and the GAT-only and Transformer variants also exceed the full model. Hence, on these benchmarks, **neither the protocol graph**

TABLE 6. COMPONENT ABLATION (CHUNK-SCORING ENCODER): TEST MACRO-F1 (VALIDATION-TUNED THRESHOLD) / AUROC, MEAN OVER SEEDS; TRAINABLE PARAMETERS; T4 TRAINING TIME OF SEED 0 ON A2A; RESIDUAL-AWARE LIPSCHITZ BOUND L_f OF THE SEED-0 A2A MODEL.

Variant	A2A sandbox	MCPTox	Params	Train	L_f
Full (SSM+GATv2, graph)	$0.952_{\pm.002}$ / 0.996	$0.731_{\pm.095}$ / 0.850	4.10 M	1187 s	5.4×10^{6}
SSM only (no GATv2)	$0.961_{\pm.015}$ / 0.997	$0.698_{\pm.137}$ / 0.834	2.35 M	842 s	3.6×10^{5}
GATv2 only (no SSM)	$0.954_{\pm.001}$ / 0.996	$0.795_{\pm.014}$ / 0.908	2.35 M	429 s	4.0×10^{2}
Flat read-out (no graph)	$0.960_{\pm.010}$ / 0.997	$\mathbf{0.841}_{\pm.003}$ / **0.945**	4.10 M	495 s	3.7×10^{6}
Transformer for SSM	$0.956_{\pm.000}$ / 0.996	$0.800_{\pm.116}$ / 0.935	4.46 M	438 s	∞
No Lipschitz penalty	$0.955_{\pm.001}$ / 0.996	$0.785_{\pm.027}$ / 0.898	4.10 M	960 s	3.6×10^{8}

nor the selective SSM is shown to help; on payload attacks the chunk scorer carries the signal and the downstream architecture mainly adds variance. We report the full model as the pre-registered configuration rather than switching to the flat read-out after seeing these test results. The Lipschitz regulariser lowers the residual-aware bound by about $66\times$ on A2A ($3.6\times10^{8} \rightarrow 5.4\times10^{6}$) without changing detection, but both values are vacuous, and the bound is infinite for the Transformer variant because softmax attention is not globally Lipschitz [27].

8. Adaptive Attacks and Robustness

A smoothing-based filter must be judged against an attacker who adapts to it, not against static inputs [39], [40]. We therefore built the attack surface into the artifact. Each attack targets a different component: white-box projected gradient descent in the payload-embedding space with Expectation-over-Transformation over the smoothing noise [41] (E1); Greedy Coordinate Gradient token search that keeps the malicious tool call fixed, to measure how far text-level evasion exceeds the feature-space budget [42] (E2); an attacker-LLM payload rewrite under a fixed query budget (E3); graph-structural evasion that splits an attack across agents or windows (E4); and manipulation of the Hedge controller's observed losses (E5).

We report here only what we measured, on CPU, against the untrained reference model. The embedding-PGD attack (E1), run on 16 synthetic inputs with 40 steps and 4 EoT samples, flips 0% of predictions at $\varepsilon \in \{0.01, 0.05\}$, 6.3% at $\varepsilon = 0.10$ and 12.5% at $\varepsilon = 0.20$: inside the certified radius the detector is empirically stable, and the first failures appear only past it, consistent with the certificate of §5. This is an implementation check on synthetic inputs, not a benchmark attack-success rate; the GCG, LLM-rewrite, graph-evasion and Hedge-manipulation attacks are implemented in the artifact but were not run against the trained benchmark models. Given §7.3, the most informative adaptive test is a paraphrase of the templated attack text, which would probe the lexical baseline and the detector alike; we have not run it.

9. Discussion and Limitations

MambaGuard is a *detector*, complementary to design-level defences like CaMeL and MELON that prevent injection outcomes by sandboxing; we view detector+design composition as the deployment target. **The main limitation is empirical.** Chunk scoring closes most of the gap to a bag-of-words classifier, but not all of it: the detector beats the lexical baseline only in AUROC on MCP-SafetyBench, and the protocol graph and selective SSM are not shown to help (§7.7). Under held-out templates both models degrade, and neither generalises across attack families (§7.4). Its clearest success, the A2A sandbox, is our own synthetic data, generated from one template per finding. The benchmarks themselves limit what can be concluded: templated attacks, small benign classes (3–24 test episodes on three of them) and few recorded models on AgentDojo. A convincing case for a structural detector needs benchmarks with paraphrased attacks, family-level held-out splits and recorded multi-agent traffic. The flat read-out's MCPTox result (macro-F1 0.84, AUROC 0.945) suggests that a simpler chunk-scoring classifier may suffice for payload attacks; we have not evaluated it as a pre-registered configuration. Other limitations: **Feature vs payload.** The smoothing certificate holds in the payload-embedding feature space; the erase-and-check composition partially bridges but does not close the gap to text-level perturbations. **Loose Lipschitz.** The sound residual-aware bound is $37\times$ the empirical lower bound at $c = 0.5$, is vacuous ($\geq 7 \times 10^{4}$) on every trained model, and the simpler block-product bound is not a bound at all; tighter data-dependent estimates are future work. **Small certified radii.** With $n = 500$ smoothing samples no radius above $\approx 2.2\sigma$ can be certified, and the measured radii (median 0.05–0.34 in the normalised payload-feature space) are not tied to any text-edit budget. **Encoder cost.** Chunk scoring makes payload encoding, not the detector, the throughput bottleneck (325–887 messages per second on one T4). **Attacker capabilities.** The threat model excludes host compromise, cryptographic-signature forgery and arbitrary new-agent injection; these capabilities need orthogonal defences. Encrypted and post-quantum traffic, and differentially private payload embeddings, are future work.

10. Related Work

Agent-protocol security monitors. ClawGuard [9] enforces deterministic tool-access policies. AgentArmor [8] reconstructs program-dependency graphs from runtime traces. DataSentinel [18] is a game-theoretic detection framework published at IEEE S&P 2025. None observes the multi-protocol interaction graph or reports a certificate.

Design-level defences. CaMeL [13] prevents injection outcomes by design. MELON [14] provides provable guarantees against indirect prompt injection in ICML 2025. Progent [15] adds programmable privilege control. The Task Shield [10] enforces task alignment. These are complementary to our detector.

Certified robustness. Tsuzuku *et al.* [31] introduced Lipschitz-margin training; Cohen *et al.* [30] introduced randomised smoothing. Bojchevski *et al.* [34] developed sparse smoothing for graphs; Schuchardt *et al.* [35], [36] extended

to collective and hierarchical variants. Kumar *et al.* [33] gave erase-and-check for LLM prompt perturbations.

Mamba / SSM stability. Halloran *et al.* [21] prove Lyapunov stability for Mamba; a parallel 2024 preprint explores adversarial robustness of deep SSMs [22].

Attention Lipschitz. Kim *et al.* [27] prove that dot-product attention is not globally Lipschitz; LipschitzNorm [26] is the standard remedy for graph-attention networks.

Adaptive-attack methodology. Nasr *et al.* [39] bypassed twelve recent prompt-injection defences; Tramèr *et al.* [40] wrote the methodological reference.

Game theory. Balcan *et al.* [38] prove regret against the best fixed commitment in hindsight for repeated Stackelberg games under unknown attacker types.

11. Conclusion

We presented MambaGuard, a streaming detector for multi-protocol LLM-agent traffic that couples a selective SSM with a Lipschitz-normalised temporal GATv2 layer and ships two type-checked certificates and a corrected online no-regret bound. We corrected the online step size, made the composition theorem unit-consistent, and showed that a block-product Lipschitz bound is unsound once residual paths are counted. Measured with three seeds on one GPU, scoring payloads at the level of tool descriptions and sentences was the decisive design choice, raising macro-F1 by 0.20–0.38 over one embedding per message on the payload benchmarks. The detector separates structural A2A attacks (macro-F1 0.952, AUROC 0.996) and approaches but does not surpass a bag-of-words baseline on payload attacks, and neither model generalises when whole attack families are held out. Current agent-security benchmarks are largely solvable from attack-template words; held-out-template evaluation, which we release with the artifact, should accompany any claim that a detector generalises. Code, weights, the adaptive-attack harness, and a reproduction script that regenerates every table and figure are released under an anonymised mirror.

References

[1] X. Hou *et al.*, "Model Context Protocol: Landscape, security threats, and future research directions," arXiv:2503.23278, 2025.

[2] Y. Yang *et al.*, "MCPSecBench: A systematic security benchmark and playground for testing model context protocols," arXiv:2508.13220, 2025.

[3] Qualys, "Anthropic MCP Inspector remote code execution vulnerability (CVE-2025-49596)," Jul. 2025.

[4] A. Lotfi, M. M. Rahman, I. Karim, E. Bertino, "A2ABreak: Systematic security analysis of the A2A protocol," arXiv:2609.10871, 2026.

[5] T. Zong *et al.*, "MCP-SafetyBench: A benchmark for safety evaluation of LLMs with real-world MCP servers," ICLR 2026, arXiv:2512.15163.

[6] Z. Wang *et al.*, "MCPTox: A benchmark for tool poisoning attack on real-world MCP servers," AAAI 2026, arXiv:2508.14925.

[7] Z. Habler *et al.*, "Building a secure agentic AI application leveraging Google's A2A protocol," arXiv:2504.16902, 2025.

[8] P. Wang *et al.*, "AgentArmor: Enforcing program analysis on agent runtime trace to defend against prompt injection," arXiv:2508.01249, 2025.

[9] W. Zhao *et al.*, "ClawGuard: Securing LLM agents through deterministic tool-access policies," arXiv:2604.11790, 2026.

[10] "The Task Shield: Enforcing task alignment to defend against indirect prompt injection in LLM agents," arXiv:2412.16682, 2024.

[11] E. Debenedetti *et al.*, "AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents," NeurIPS 2024.

[12] Q. Zhan *et al.*, "InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents," Findings of ACL 2024.

[13] E. Debenedetti *et al.*, "Defeating prompt injections by design (CaMeL)," arXiv:2503.18813, 2025.

[14] K. Zhu *et al.*, "MELON: Provable defense against indirect prompt injection attacks in AI agents," ICML 2025, PMLR 267:80310–80329.

[15] T. Shi *et al.*, "Progent: Programmable privilege control for LLM agents," arXiv:2504.11703, 2025.

[16] Meta AI, "PromptGuard: A compact classifier for prompt injection," 2024.

[17] "PIGuard: Prompt injection detection," arXiv:2411.XXXXX, 2024.

[18] Y. Liu *et al.*, "DataSentinel: A game-theoretic detection of prompt injection attacks," IEEE S&P 2025, arXiv:2504.11358.

[19] A. Gu, T. Dao, "Mamba: Linear-time sequence modeling with selective state spaces," arXiv:2312.00752, 2023.

[20] A. Gu *et al.*, "On the parameterization and initialization of diagonal state space models," NeurIPS 2022.

[21] J. T. Halloran *et al.*, "Mamba state-space models are Lyapunov-stable learners," TMLR 2025, arXiv:2406.00209.

[22] "Exploring adversarial robustness of deep state space models," arXiv:2406.05532, 2024.

[23] T. Wang *et al.*, "NetMamba: Efficient network traffic classification via pre-training unidirectional Mamba," IEEE INFOCOM 2025.

[24] S. Brody, U. Alon, E. Yahav, "How attentive are graph attention networks?," ICLR 2022.

[25] D. Xu *et al.*, "Inductive representation learning on temporal graphs," ICLR 2020.

[26] G. Dasoulas, K. Scaman, A. Virmaux, "Lipschitz normalization for self-attention layers with application to graph neural networks," ICML 2021.

[27] H. Kim, G. Papamakarios, A. Mnih, "The Lipschitz constant of self-attention," ICML 2021.

[28] W. W. Lo *et al.*, "E-GraphSAGE: A graph neural network based IDS for IoT," IEEE/IFIP NOMS 2022.

[29] Z. Wu *et al.*, "RTIDS: A robust transformer-based approach for IDS," IEEE Access, vol. 10, 2022.

[30] J. M. Cohen, E. Rosenfeld, J. Z. Kolter, "Certified adversarial robustness via randomized smoothing," ICML 2019.

[31] Y. Tsuzuku, I. Sato, M. Sugiyama, "Lipschitz-margin training," NeurIPS 2018.

[32] M. Fazlyab *et al.*, "Efficient and accurate estimation of Lipschitz constants for deep neural networks," NeurIPS 2019.

[33] A. Kumar *et al.*, "Certifying LLM safety against adversarial prompting," COLM 2024, arXiv:2309.02705.

[34] A. Bojchevski, J. Gasteiger, S. Günnemann, "Efficient robustness certificates for discrete data: Sparsity-aware randomized smoothing for graphs, images and more," ICML 2020.

[35] J. Schuchardt *et al.*, "Localized randomized smoothing for collective robustness certification," ICLR 2023.

[36] J. Schuchardt *et al.*, "Hierarchical randomized smoothing," NeurIPS 2023.

[37] N. Cesa-Bianchi, G. Lugosi, *Prediction, Learning, and Games*. Cambridge Univ. Press, 2006.

[38] M.-F. Balcan, A. Blum, N. Haghtalab, A. D. Procaccia, "Commitment without regrets: Online learning in Stackelberg security games," ACM EC 2015.

[39] M. Nasr *et al.*, "The attacker moves second: Stronger adaptive attacks bypass defenses against LLM jailbreaks and prompt injections," USENIX Security 2026, arXiv:2510.09023.

[40] F. Tramèr *et al.*, "On adaptive attacks to adversarial example defenses," NeurIPS 2020.

[41] A. Athalye, N. Carlini, D. Wagner, "Obfuscated gradients give a false sense of security," ICML 2018.

[42] A. Zou *et al.*, "Universal and transferable adversarial attacks on aligned language models (GCG)," arXiv:2307.15043, 2023.

[43] I. Sharafaldin *et al.*, "Toward generating a new intrusion detection dataset," ICISSP 2018.

[44] E. C. P. Neto *et al.*, "CICIoT2023," *Sensors*, vol. 23, 2023.

[45] N. Moustafa, J. Slay, "UNSW-NB15," MilCIS 2015.

Appendix A. Mathematical Preliminaries

Standard restatement of S4D-Real Mamba, dynamic graph attention and TGAT time encoding; omitted from body for space. Full derivations in the anonymised artifact (file `docs/preliminaries.pdf`).

Appendix B. Proof of Lemma 1 (Local SSM Lipschitz)

Under Assumption 1, the state recurrence $h_t = \sum_{k=0}^{t-1}(\prod_{j=t-k+1}^{t}\bar{A}_j)\bar{B}_{t-k}m_{t-k}$ gives $\|h_t(x) - h_t(x')\| \le \sum_{k\ge 0}\rho^k\beta\|m_{t-k} - m'_{t-k}\| = \beta/(1-\rho)\|x - x'\|_{\infty,t}$. Multiplying by $\gamma = \sup\|C_t\|$ for the output, and by the Hadamard-gate factor $(1 + L_\sigma\kappa_g)$ since $\|fg\|_{\text{Lip}} \le \|f\|_\infty L_g + \|g\|_\infty L_f$ with $\|f\|_\infty, \|g\|_\infty \le \gamma\beta X_{\max}/(1-\rho)$, bounded by $X_{\max}$, yields the first term of (5). The second term absorbs the Jacobians of $\bar{B}_t$, C_t and Δ_t with respect to the input; applying submultiplicativity once more gives $\gamma\beta X_{\max}(M_B + M_C + M_\Delta)/(1-\rho)^2$. The depthwise causal convolution is folded into β via its unrolled Toeplitz norm. □

Appendix C. Proof of Theorem 2 (Composed Certificate)

Partition the horizon into rounds where $\mathcal{C}_\varepsilon$ holds and rounds where it does not. On $\mathcal{C}_\varepsilon$ the detector's prediction, hence the defender's action, hence the per-round utility, is unchanged under attack and therefore at least V^* in expectation (by the Stackelberg optimality under the zero-sum or fixed-commitment restriction). On $\neg\mathcal{C}_\varepsilon$ the per-round loss is at most B. The Hedge regret $R_T \le B\sqrt{(T/2)\ln N}$ divided by T gives the second penalty. The α_{MC} term is the probability that the Clopper-Pearson lower bound fails and the certified event is reported in error. □

Appendix D. Exact SiLU Lipschitz Constant

$\frac{d}{du}\text{SiLU}(u) = \sigma(u)(1+u(1-\sigma(u)))$; numerical solution of $\text{SiLU}''(u) = 0$ yields $u^* \approx 2.3994$, $\text{SiLU}'(u^*) \approx 1.0998$.

Appendix E. Hyperparameters

$N = 16$, $d_z = 256$, expansion $E = 2$, kernel width $k = 4$, 4 stacked SSM layers; $H = 4$ heads, $d_T = 64$; AdamW lr 3×10^{-4}, weight decay 10^{-2}, cosine schedule with 200 warm-up steps, batch 32, class-balanced sampling, early stopping on validation macro-F1 (patience 5) with at most 15/20/30/30/20 epochs (A2A/MCPTox/MCP-SafetyBench/AgentDojo/InjecAgent); focal loss; $\lambda_L = 10^{-3}$ unless otherwise noted; fp32 on one NVIDIA T4; seeds 0, 1, 2 (ablations and held-out folds 0, 1; A2A held-out folds 0). Payload encoder: chunks of ≤ 48 words or one per tool, at most 64 per message; frozen `sentence-transformers/all-MiniLM-L6-v2` chunk embeddings (384-d) and hashed log-count word uni/bigram vectors (512 buckets), both L2-normalised; message vector $[\text{mean}, \max \text{ of embeddings}, \max \text{ of lexical vectors}]$ ($d_p = 1{,}280$); chunk scorer $896 \to 128 \to 128$ with GELU, max+mean pooled. A2A sandbox: 200 attack/benign pairs per finding (held-out folds: 100, at most 10 epochs). Held-out AgentDojo folds: GPT-4o recorded runs, at most 100/25/40 attack episodes per family for train/val/test. Smoothing on benchmarks: 100 test windows, $n_0 = 32$, $n = 500$, $\alpha_{MC} = 10^{-3}$, $\sigma \in \{0.05, 0.10, 0.25\}$; on synthetic inputs: $n = 1000$. Sparse smoothing: $p_+ = p_- = 0.05$, $n = 1000$.

Appendix F. Protocol-Benchmark Canonicalisation

Loaders for MCPTox, MCP-SafetyBench, MCPSecBench and the A2A-sandbox are in `mambaguard/data/protocol_traces/`; `reproducibility/PROTOCOL_BENCHMARKS.md` gives the pinned source commits, the episode construction for each benchmark, expected counts, and the shortcut baselines. Server source files are parsed statically and never executed. MCPTox and MCP-SafetyBench carry no license and are read from the user's own clone, not redistributed.

Appendix G.
Graph-Edit Certified Budgets

Full distribution of (k_a, k_d) over the six benchmarks and the sensitivity to p_+, p_-. See the artifact for CSV.

Appendix H.
Erase-and-Check Composition

The feature-space smoothing certificate can be composed with the erase-and-check certificate [33], which certifies against adversarial suffixes, insertions and infusions of up to d tokens. We describe the composition here as the route to a payload-level guarantee; a quantitative evaluation is future work.

Appendix I.
Network-IDS Transfer (Not in Headline Claims)

Classical network-IDS corpora (CSE-CIC-IDS2018, CIC-IoT2023, UNSW-NB15) are supported by the same canonicaliser and loaders, but a transfer evaluation is out of scope for this paper, which targets agent-protocol threats.

Appendix J.
Ethics Considerations

The adaptive-attack code in §8 could in principle be adapted to attack production MCP servers. We release the scaffolding needed to reproduce the paper but not working exploits against live third-party services. All protocol traces are synthetic or from public benchmarks; no live third-party agent deployment was targeted and no human-subject data was collected. We do not include CVE identifiers in the submission.

Appendix K.
Generative AI Usage

The benchmarks used in this paper are either public (MCPTox, MCP-SafetyBench, MCPSecBench, AgentDojo, InjecAgent) or generated deterministically from a fixed random seed with no model in the loop (the A2A-sandbox); no closed model was queried to produce the data or the results reported here. Generative AI was used for editorial purposes in this manuscript, and all outputs were inspected by the authors to ensure accuracy and originality.

Appendix L.
Artifact Availability

An anonymised artifact is available at https://anonymous.4open.science/r/mambaguard-sp2027 (identifier filled in at submission). It contains the detector, the smoothing, sparse-smoothing and composed certificates, the Hedge controller, the adaptive attacks, the training and evaluation harness, all configuration files, and 43 tests. `scripts/reproduce_all.sh` runs the tests and regenerates every synthetic-input result in about ten minutes on a CPU. `kaggle/run_kaggle_study.py --study all` reruns the whole benchmark study (all seeds, the lexical baseline, encoder and architecture ablations, held-out-template folds, certificates, latency) on one GPU in about seven hours. Recorded outputs of the reported run are in `reproducibility/kaggle_gpu_v2/` (every run, prediction, certificate and latency file) and `reproducibility/kaggle_results_v2.json`; `reproducibility/kaggle_gpu/` holds an earlier run with one embedding per message, and `reproducibility/experiments_sp2027.json` the synthetic-input checks. The artifact is frozen at the submission deadline and is not updated during review, per the S&P 2027 artifact policy.